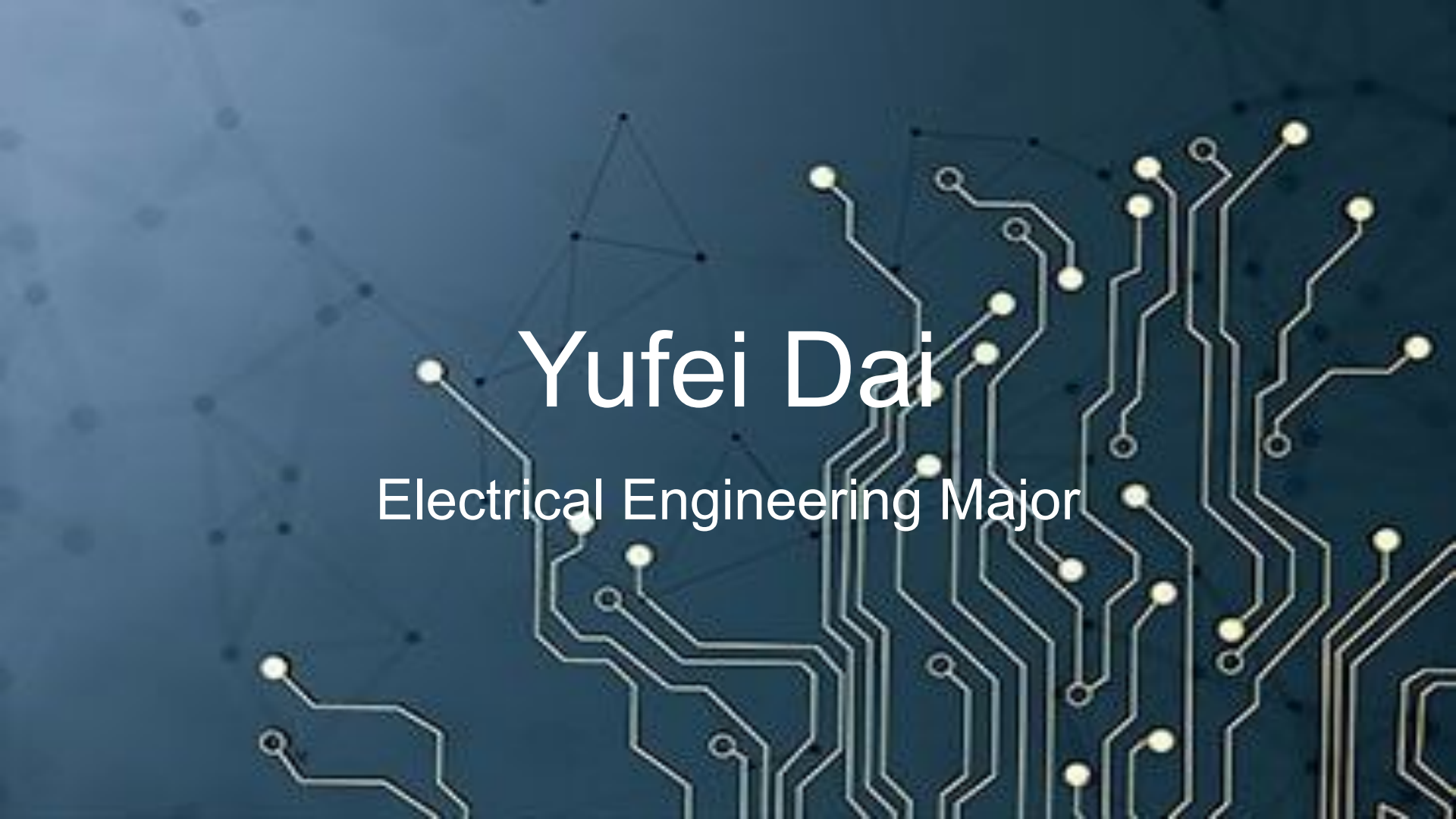


The background is a dark blue gradient. On the right side, there are intricate white circuit traces, some of which terminate in small, glowing yellow circular nodes. On the left side, there is a faint, light blue network graph consisting of interconnected dots and lines. The overall aesthetic is technological and digital.

Yufei Dai

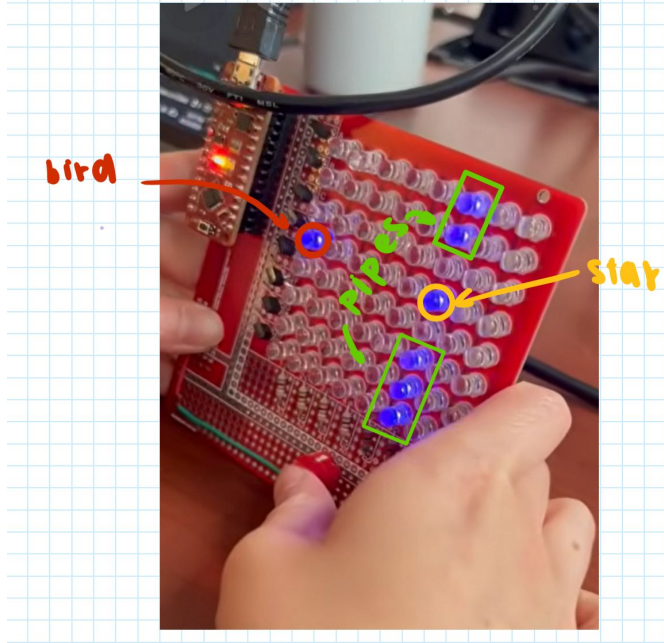
Electrical Engineering Major

Portfolio Walkthrough (15 min)

For the complete for all programming projects follow this link to github:
<https://github.com/daiyuf25/Yufei-s-Portfolio>

Flappy Bird

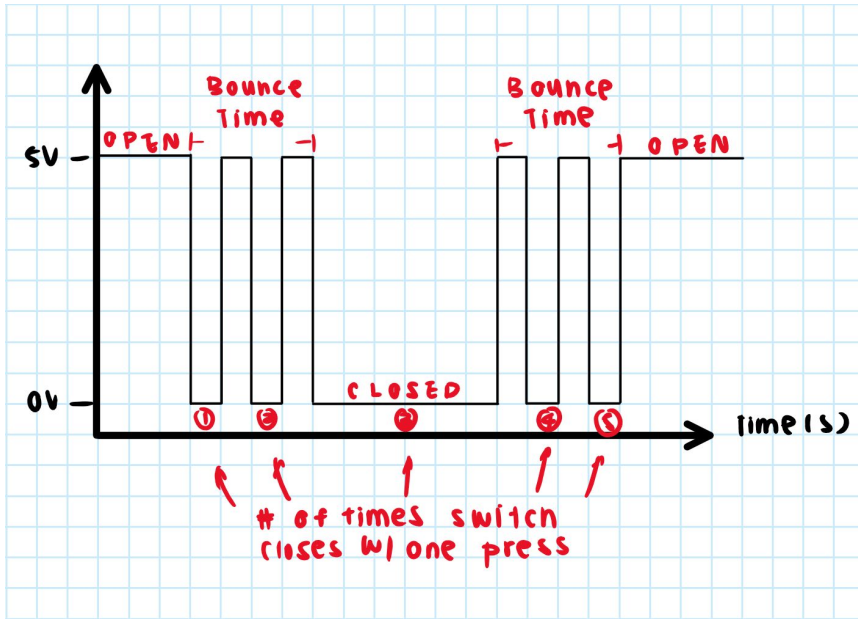
Overview: I recreated flappy bird on an 8x8 LED display board. See here for [demo!](#)



Debouncing the Switch

The push button has a bouncing phenomenon when it gets pressed where it vibrates each time before reaching each steady state.

Signals Arduino Receives from Push Button



The arduino code with register this one button press as 5 different ones.

Debouncing the Switch Using Code

```
const int DEBOUNCE_DELAY = 100;
```

```
static int counter = 0;           // count (debounced) button presses
static byte button_state = HIGH;
static byte last_reading = HIGH;
static long last_reading_change = 0;
static char message[50];         // buffer for sprintfs
```

Debouncing the Switch Using Code

```
byte reading = digitalRead(BUTTON);

unsigned long now = millis();

// Ignore button_state changes within DEBOUNCE_DELAY milliseconds of the last
// reading change, otherwise accept.

if (now - last_reading_change > DEBOUNCE_DELAY) {

    if (reading == LOW && button_state == HIGH) {

        sprintf(message, "accepted [%d], %ld ms since last\n", ++counter,

            now - last_reading_change);

        Serial.print(message);

        //if the button is pressed move the bird up by one
    }

    button_state = reading;
```

Checks to see if enough time has passed since that last time accepted input from the push button pin



Checks to see if the button was pressed



Debouncing the Switch Using Code

```
// For educational purposes, mention when we ignore a reading change. You
// (probably) don't need this code when you incorporate switch bouncing into
// your cool program.

else if (reading == LOW && last_reading == HIGH) {

    sprintf(message, "ignored, %ld ms since last\n", now - last_reading_change);

    Serial.print(message);

}

// Prepare for next loop

//rest reading to high if button is released

if (reading != last_reading) last_reading_change = now;

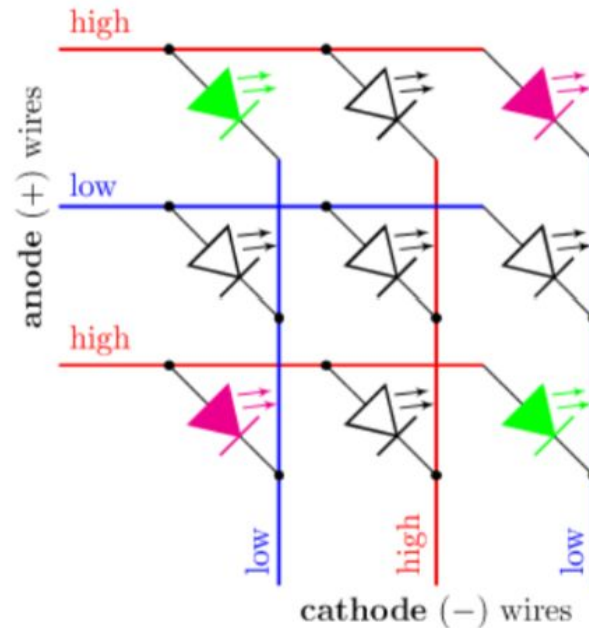
last_reading = reading;
```

Updates the last time the Arduino accepted the signal from the input pin connected to the push button to now



Time Division Multiplexing

Problem with Displaying on LED Lights all at once



Programing Arduino to Implement Time Division Multiplexing

Pseudocode for display() method

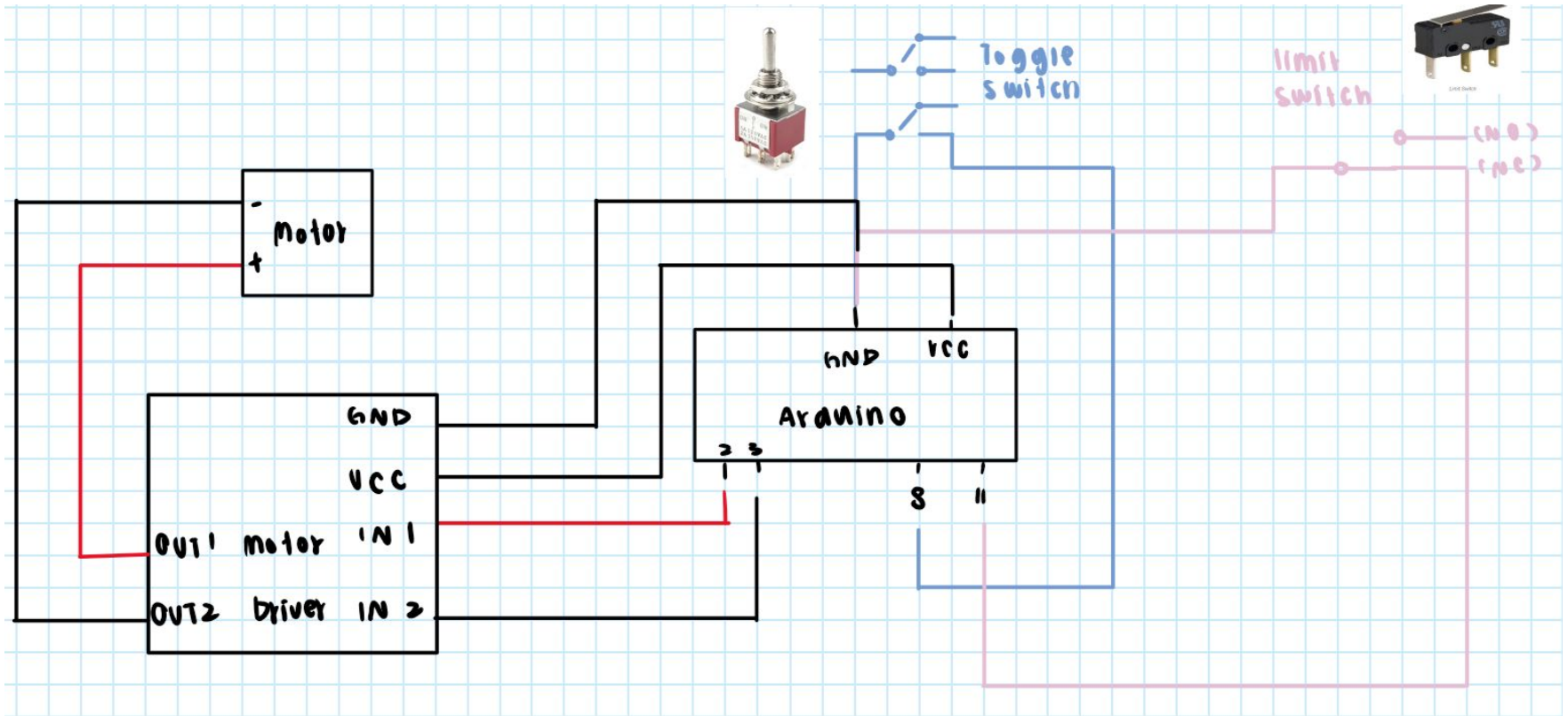
```
for each row (+) wire
  for each column (−) wire
    look up in pattern whether this LED should be on or off
    if LED should be on, activate column (−) wire, else deactivate it
  end for
  activate row (+) wire
  wait a short time
  deactivate row (+) wire
end for
```

Useless Box

Overview: The useless box is a toy where you flip a switch on the box and a finger in the box flips the switch back. I changed the Arduino code to give my useless box a personality. This [video](#) is a demo of the the useless box I made.



Electrical Schematic



Code Overview

Setting up the Arduino Pins

```
void setup() {  
  Serial.begin(115200);  
  Serial.println("Setup complete");  
  
  // TO DO:  
  // initialize input and output pins  
  pinMode(TOGGLE, INPUT_PULLUP);  
  pinMode(LIMIT, INPUT_PULLUP);  
  pinMode(two, OUTPUT);  
  pinMode(three, OUTPUT);  
  
  // initialize your initial states  
  state = STOP;  
}
```

Loop() Structure

```
switch (state) {  
  case STOP:  
    // code  
    break;  
  
  case FORWARD:  
    // code  
    break;  
  
  default:  
    // code  
    break;  
}
```

Stop Case

```
case STOP: // case for STOP
  // TO DO: Set your outputs for the motor to stop
  digitalWrite(two, LOW);
  digitalWrite(three, LOW);

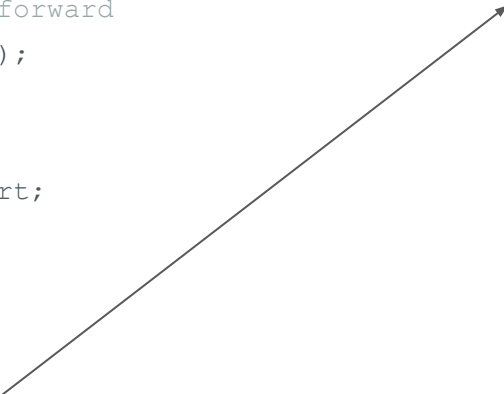
  // TO DO: Decide whether to change states and update the state variable accordingly
  if (digitalRead(TOGGLE) == LOW){
    state = FORWARD;
  }
  break;
```



Forward/Reverse Case

```
case FORWARD: // case for forward
    Serial.println("FORWARD");

    int now = millis();
    int interval = now - start;
    Serial.print("start ");
    Serial.println(start);
    Serial.print("now ");
    Serial.println(now);
```



```
if (0 <= interval && interval <
INTERVAL_SHY) {
    Serial.println("SHY_CASE");

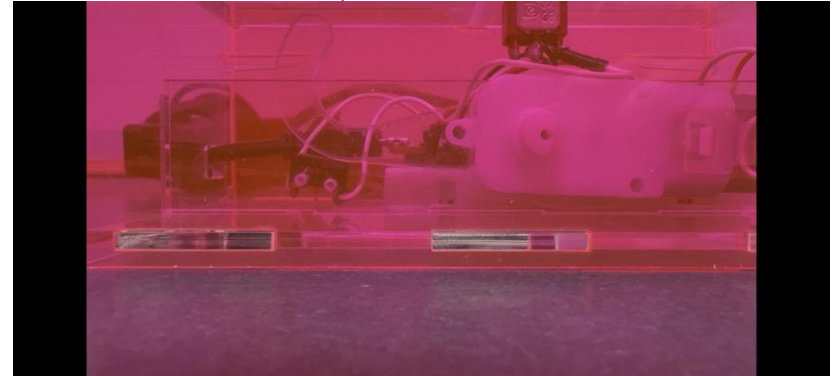
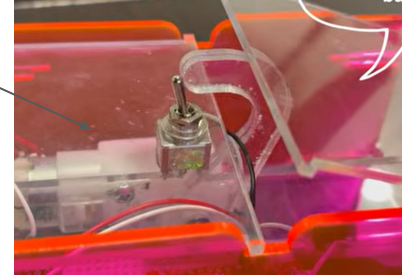
    //start moving slowly
    for (int i = 0; i <= 15; i++) {
        digitalWrite(three, LOW);
        digitalWrite(two, HIGH);
        delay(10);
        digitalWrite(three, LOW);
        digitalWrite(two, LOW);
        delay(30);
        Serial.println("forward");
    }
```

Forward/Reverse Case

```
//continue moving
for (int i = 0; i <= 15; i++)
    digitalWrite(three, LOW);
    digitalWrite(two, HIGH);
    delay(20);
    digitalWrite(three, LOW);
    digitalWrite(two, LOW);
    delay(10);
    Serial.println("forward");
}

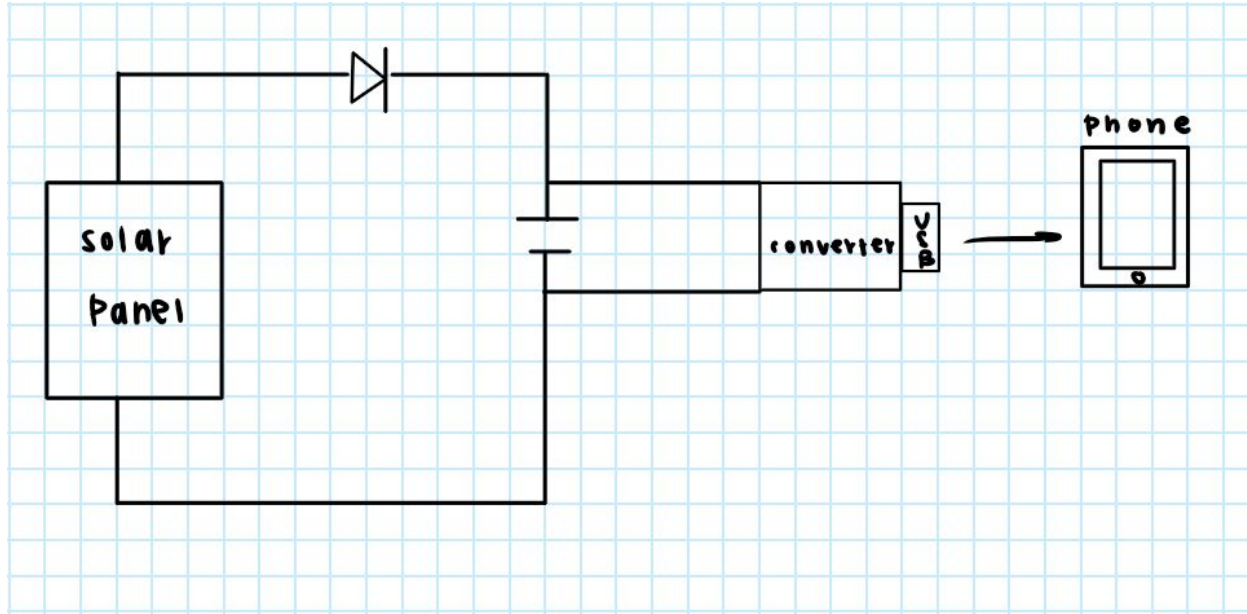
digitalWrite(three, LOW);
digitalWrite(two, HIGH);
Serial.println("forward");
delay(500);
```

```
if (digitalRead(TOGGLE) == HIGH){
    digitalWrite(two, LOW);
    digitalWrite(three, HIGH);
    delay(1000);
    if (digitalRead(LIMIT) == HIGH){
        Serial.println("stop");
        state = STOP;
    }
}
```



Solar Panel Phone Charger

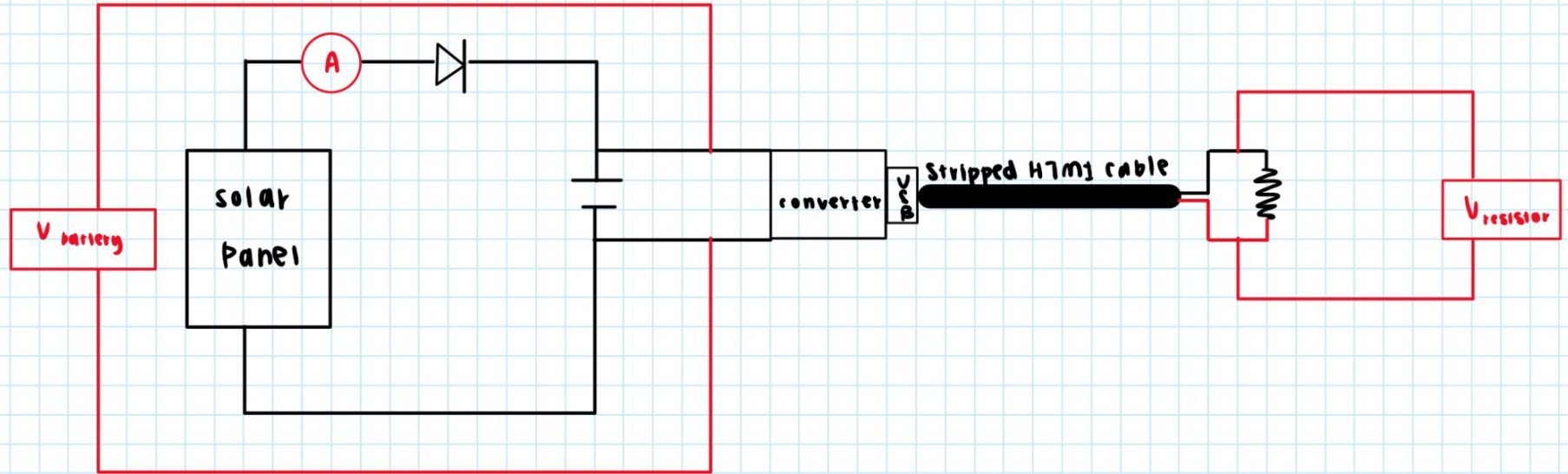
Schematic:



Picture:



Finding True Converter Efficiency



Method

	Battery			Resistor			Efficiency
	Voltage	Current	Power	Voltage	Resistance	Power	
82Ω	3.9	0.12	0.468	5.1	81	0.321	0.686
27Ω	3.81	0.28	1.0668	5.03	26	0.973	0.912
13.5Ω	3.05	0.7	2.135	4.86	13	1.814	0.851

$$P_{\text{Battery}} = i_{\text{Battery}} * v_{\text{Battery}} = P_{\text{in}} \qquad P_{\text{Resistor}} = \frac{v_{\text{Resistor}}^2}{R} = P_{\text{out}}$$

$$\text{Efficiency} = \frac{P_{\text{out}}}{P_{\text{in}}}$$

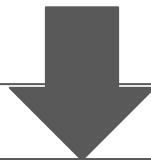
Guess the Picture with Python

Overview: This code reveals one small section of a picture per round to a user who tries to guess what is depicted in the picture. The code makes an API call to ChatGPT to verify the user's answers. This helps account for the variability of user's responses that may not exactly match the answer given by the coder.



Revisions

```
def get_answer_validation(self, guess):
    chat_completion = openai.chat.completions.create(
        messages=[{
            "role": "user",
            "content": "My friends are playing guess the image. The correct answer is " + self._correctanswer + "."
            " Would " + str(guess) + " be a valid answer too? Please be lenient and please just respond yes or no."
        }],
        model="gpt-4",
    )
```



```
def get_answer_validation(self, guess):
    chat_completion = openai.chat.completions.create(
        messages=[{
            "role": "user",
            "content": "My friends are playing guess the image. The correct answer is " + self._correctanswer + ". "
            "Would " + str(guess) + " be a valid answer too? Please be lenient but don't allow answers that are not specific enough. "
            "Please just respond yes or no."
        }],
        model="gpt-6-astra",
    )
```

Answer Validation w/ GPT-6 Astra



Answer Validation Results

Agreed with ChatGPT	Disagreed from ChatGPT
✓ Sweet gummy candy with sour coating shaped like a kid	✗ gummy candy
✓ suor patch kdis	
✗ candy	
✗ colorful stuff	

Game Demos

Demo 1



Demo 2



Using the API Call

- Code in Guess the Picture.py (main program)
- ImageGuessingAssistant.py

```
assistant = ImageGuessingAssistant(pictures[name_image_to_guess])
```

```
def __init__(self, answer):  
    self._correctanswer = answer  
    self._hints = []
```

Create an image object by passing in the answer for the current picture in play

```
while len(puzzle_pieces_list) != 1 and assistant.get_answer_validation(guess) == False:
```

Calling the method `get_answer_validation` in `ImageGuessingAssistant.py`

Using the API Call

Correct answer



```
def get_answer_validation(self, guess):
    chat_completion = openai.chat.completions.create(
        messages=[{
            "role": "user",
            "content": "My friends are playing guess the image. The correct answer is " + self._correctanswer + ". "
            "Would " + str(guess) + " be a valid answer too? Please be lenient but don't allow answers that are not specific enough. "
            "Please just respond yes or no."
        }],
        model="gpt-6-astra",
    )
    response_str = chat_completion.choices[0].message.content
    if response_str.strip(".").lower() == "yes":
        return True
    return False
```

User guess



Return whether or not the user's guess is valid



Thank You!